# WashMate Campus
# Product and Implementation Guide

WashMate Campus Team

2026-05-31

# Contents

# 1 Document Purpose

This document describes the implemented WashMate Campus product from a product, frontend, backend, data model, testing, and delivery perspective. It is written from repository facts instead of from a target architecture that has not been built.

The most important architectural fact is that WashMate Campus is not a traditional web frontend calling a long-running Python HTTP backend. The primary user-facing product is the React, TypeScript, Vite, and Capacitor mobile app under `frontend/`. Its core business logic runs inside the APK as TypeScript modules under `frontend/src/api/`. The Python package under `backend/` is a reference implementation and test reference. It defines behavior, validates rules, and supports integration tests.

# 2 Product Overview

## 2.1 Problem and Users

WashMate Campus targets campus shared-laundry scenarios. A student usually needs to decide whether clothes should be washed today, which clothes can be washed together, whether a dryer is safe, which dormitory machines are available, and how much time and money the session will cost. These decisions depend on wardrobe memory, clothing care requirements, machine status, weather, personal constraints, and local campus laundry rules.

The product turns those inputs into a mobile workflow:

1. The user maintains a local wardrobe with materials, colors, care tags, photos, wear counts, wash records, and user notes.
2. The user records worn outfits or adds items to the dirty basket.
3. The app reads campus context after the user configures a dormitory building.
4. The planning module splits laundry into safe buckets, assigns machine programs when possible, and estimates wait time, duration, and cost.
5. The report screen presents the plan as actionable steps, risk notes, savings notes, and cost lines.

## 2.2 Implemented User-Facing Scope

The current mobile surface includes Today, Outfit Wiki, Record Outfit, Wardrobe, Add Clothing, Clothing Detail, Dirty Basket, Laundry Room, Machine Detail, Plan Detail, Report, and Profile screens. The main navigation and state management live in `frontend/src/App.tsx`; reusable shell UI lives in `frontend/src/components/`; product copy and static fixtures live in `frontend/src/data/`.

The implemented product supports:

- Manual clothing entry and local photo attachment.
- User-triggered ModelHub image or text recognition.
- Local wardrobe CRUD, wear-count tracking, and dirty-basket state.
- Outfit logging and outfit recommendation support.

- Dormitory selection, live campus machine context, and weather context.
- Rule-based laundry planning with wash buckets, dryer recommendations, warnings, cost, and duration.
- Completed-laundry history with weekly report summaries and undo support.
- Structured report generation for display without natural-language reverse parsing.

The latest mobile scripts also improve several execution details. Add Clothing now shows the missing ModelHub configuration reason next to recognition buttons. Outfit Wiki tells the user when bottoms are missing and provides an add-clothing action. Laundry Room shows machine ids and highlights machines recommended by the current plan. After the user completes a laundry plan, the app stores a local completion record so the report still has useful weekly history after the dirty basket is cleared.

# 3 Repository and Stack

## 3.1 Technology Stack

| Layer | Main technology | Implemented role |
|---|---|---|
| Mobile UI | React 18, TypeScript, Vite | Mobile screens, navigation, forms, local state, and user-facing interaction. |
| APK shell | Capacitor 7, Android Gradle | Android WebView packaging, app identity, native sync, and debug or release APK build path. |
| APK services | TypeScript modules in `frontend/src/api/` | Wardrobe, dirty basket, completed-laundry records, campus context, laundry planning, report generation, frequency advice, recognition request logic, local storage, and mobile summaries. |
| Python reference | Python 3.12, dataclasses, unittest | Shared data models, reference extraction, wardrobe, campus, planning, report, and integration behavior. |
| External services | ModelHub/Gemini v1beta, CleverSchool, Haier, Open-Meteo | Clothing recognition, dormitory machine status, and weather. |
| Tooling | uv, npm, GitHub Actions | Dependency sync, tests, frontend build, Capacitor sync, Android build, and release automation. |

## 3.2 Key Repository Layout

```
Wash-agent/
  README.md
  AGENTS.md
  pyproject.toml
  app.py
  main.py
  backend/
    shared/models.py
    clothing_extraction/
    wardrobe/
    campus/
    laundry/
    reports/
  frontend/
    src/
      App.tsx
      api/
      components/
      screens/
      data/
    android/
    package.json
    capacitor.config.ts
  config/
    api_config.example.json
    machine_rules.json
  data/
    wardrobe_sample.json
    machines_mock.json
    pics/
  docs/
    structure_design.md
    c_delivery.md
    e_demo_script.md
    superpowers/plans/
  tests/
    test_*.py
  .github/workflows/
    preview.yml
    release-apk.yml
```

# 4 Full-Stack Architecture

## 4.1 Runtime Shape

The runtime is centered on the mobile app:

1. The user opens the Capacitor app.
2. React screens request a *mobile summary* or perform local actions.
3. TypeScript services in `frontend/src/api/` read local storage, compose wardrobe state,

call external services only when the user action or profile configuration requires it, and run rule-based planning/report generation.

4. When the user confirms a plan, the mobile service saves a completion snapshot, clears the dirty basket, and keeps an undo path.

5. The Python modules provide reference behavior and a test reference, but they are not required to run as an HTTP server for the normal mobile workflow.

### 4.2 Architecture Flow

```
User
  -> React / Capacitor screens
  -> frontend/src/api/mobileSummary.ts
      -> userProfile and ModelHub config
      -> local wardrobe and dirty basket
      -> campusMachineApi + weatherService
      -> frequencyAdvisor
      -> laundryPlanner
      -> reportGenerator
      -> completed laundry records
  -> mobile screens render plan, report, machines, and wardrobe state

Python backend
  -> backend/shared/models.py defines shared data models
  -> backend/* modules define reference behavior and tests
  -> tests/ verifies extraction, wardrobe, campus, planning, report, and integration
```

### 4.3 Public Boundary Principle

Frontend screens should handle user interaction only. They should not contain ModelHub request text, machine parsing rules, wardrobe persistence internals, laundry bucket rules, or report assembly rules. Those responsibilities belong to service modules or backend reference modules.

Cross-module data should pass through explicit data models. In Python, the shared model layer is `backend/shared/models.py`. In the mobile app, the matching type layer is `frontend/src/api/types.ts`. New work should not create parallel models when an existing model already describes the data.

## 5 Frontend Implementation

### 5.1 Application Shell and Navigation

`frontend/src/App.tsx` owns the active screen, browser history behavior, selected clothing and machine ids, user profile state, ModelHub configuration state, latest completed laundry record, mobile summary refresh state, and top-level handlers. It imports screens from `frontend/src/screens/` and renders the bottom navigation from `frontend/src/components/AppChrome.tsx`.

The app uses a screen id model rather than a separate router package. Parent tabs map detail pages back to their owning tab. Examples include:

- `planDetail` and `dirtyBasket` under Today.
- `addClothing` and `clothingDetail` under Wardrobe.
- `machineDetail` under Laundry Room.
- `recordOutfit` under Outfit Wiki.

### 5.2 Mobile Screens

| Screen | Responsibility |
|---|---|
| `TodayScreen.tsx` | Entry dashboard for current summary, weather, dirty-basket status, and plan entry. |
| `DirtyBasketScreen.tsx` | Lets the user select the wardrobe items that are currently waiting for laundry. |
| `PlanDetailScreen.tsx` | Shows the detailed wash plan, confirms execution in a dialog, and shows success plus undo after completion. |
| `OutfitWikiScreen.tsx` | Provides outfit classification, style summary, and recommendation support; when bottoms are missing, it asks the user to add them. |
| `RecordOutfitScreen.tsx` | Records what the user wore and increments wear counts for known wardrobe items. |
| `WardrobeScreen.tsx` | Lists local wardrobe items, supports detail entry, deletion, and clearing. |
| `AddClothingScreen.tsx` | Collects manual input, local photo selection, and optional ModelHub recognition before saving; missing recognition config is shown next to the recognition buttons. |
| `ClothingDetailScreen.tsx` | Shows one local wardrobe item, care memory, wear count, and dirty-basket action. |
| `LaundryRoomScreen.tsx` | Displays configured dormitory machine context, machine ids, queue estimates, refresh state, weather, and plan-recommended machine highlights. |
| `MachineDetailScreen.tsx` | Shows one selected machine with user-facing price and program information. |
| `ReportScreen.tsx` | Renders structured report sections, a concise now-do-this card, weekly completion history, risk notes, and savings notes. |
| `ProfileScreen.tsx` | Stores dormitory, floor, pickup time, and dryer preference; ModelHub and demo data controls live under advanced settings. |

### 5.3 Frontend Service Modules

| Module | Responsibility |
|---|---|

| | |
|---|---|
| `mobileSummary.ts` | Composes the data consumed by screens: wardrobe, dirty basket, completed-laundry history, weather, campus context, frequency advice, plan, drying plan, and report. It also completes and undoes laundry sessions. |
| `types.ts` | Defines TypeScript data models that mirror the Python shared models, plus `CompletedLaundryRecord` and `CompletedLaundrySummary`. |
| `userProfile.ts` | Loads and saves local profile values such as dormitory name, floor, pickup time, and dryer preference. |
| `modelHubConfig.ts` | Loads, saves, clears, validates, and constrains ModelHub configuration. The supported model list currently includes `gemini-3.1-pro-preview`. |
| `modelHubRecognition.ts` | Sends user-triggered image or text recognition requests and requires structured JSON output. |
| `clothingExtractor.ts` | Builds local clothing profiles and converts stored wardrobe items into planning inputs. |
| `campusTowerDirectory.ts` | Provides user-facing dormitory options while keeping provider ids inside API/config layers. |
| `campusMachineApi.ts` | Builds live campus context for a configured dormitory and normalizes machine status. |
| `weatherService.ts` | Fetches Tsinghua-area weather from Open-Meteo. |
| `pricingRules.ts` | Packages price, duration, program, and drying-context rules used by planning and display. |
| `frequencyAdvisor.ts` | Computes wash-frequency priority and recommendations from wardrobe and constraints. |
| `laundryPlanner.ts` | Splits laundry into buckets, assigns machine programs, produces warnings, and estimates cost/duration. |
| `reportGenerator.ts` | Converts the final plan and context into structured report content. |
| `outfitLogStore.ts` and `outfitWiki.ts` | Store outfit history, derive clothing pairs, and support outfit recommendation workflows. |
| `photoThumbnail.ts` and `photoFileStorage.ts` | Create and store local wardrobe photo thumbnails. |

### 5.4 Local State and Persistence

The app stores user profile, ModelHub recognition configuration, wardrobe items, dirty-basket selection, completed laundry records, outfit logs, and local photo references on the device. The design intentionally keeps normal wardrobe, basket, planning, completion history, and report workflows self-contained in the APK.

Completed laundry records are stored under `washmate.completedLaundryRecords`. Each record includes completed time, item ids, item names, cost and duration estimates, machine labels, a plan summary, and the wardrobe counters before completion. Undo uses that snapshot to restore wear counts, wash counts, and dirty-basket selection.

Only these actions access external services:

- The user triggers image or text recognition through a configured ModelHub endpoint.
- The user has configured a dormitory, and the app fetches CleverSchool or Haier machine status for that dormitory.
- The app fetches Tsinghua-area weather from Open-Meteo.

The UI must show explicit unconfigured, unavailable, loading, or error states when required input or service output is absent. It should not invent a dormitory, machine status, price, weather, material, or care result.

# 6 Backend Reference Implementation

## 6.1 Shared Models

`backend/shared/models.py` is the Python shared model layer. It defines enums and data-classes used across extraction, wardrobe, campus, planning, and report modules.

Important model groups include:

| Model group | Models |
|---|---|
| Care and risk | `RiskLevel`, `WashMethod`, `DryMethod`. |
| Machine context | `MachineType`, `MachineStatus`, `MachineTower`, `MachineInfo`, `MachineQueueEstimate`, `CampusContext`. |
| Clothing and wardrobe | `ClothingInput`, `LLMResponse`, `ClothingProfile`, `WashRecord`, `WardrobeItem`. |
| Planning | `LaundryConstraints`, `FrequencyAdvice`, `LaundryChargeLine`, `LaundryBucket`, `LaundryPlan`, `DryingStep`, `DryingPlan`. |
| Reporting | `WashReport`. |

## 6.2 Clothing Extraction

The clothing extraction package owns product/text/image normalization and reference clothing-profile extraction:

- `backend/clothing_extraction/llm_client.py` builds ModelHub/Gemini requests, image content, request text, and JSON response formats.
- `backend/clothing_extraction/product_info.py` normalizes product name, shop name, tag text, OCR text, user description, user note, and supplemental sources.
- `backend/clothing_extraction/extractor.py` converts normalized input and optional model output into a `ClothingProfile`, including materials, colors, care actions, care symbols, risks, evidence levels, missing fields, and extraction status.

When image recognition is used, the app sends one request that requires JSON output through `responseMimeType=application/json` and `responseSchema`. The extraction module should not store wardrobe state, read campus machines, decide whether the current basket should be washed, or generate reports.

### 6.3 Wardrobe Memory and Frequency Advice

`backend/wardrobe/store.py` provides `WardrobeStore`, which owns wardrobe CRUD, wash history, wear count, item validation, and JSON serialization. `backend/wardrobe/frequency_advisor.py` computes priority scores and wash-frequency recommendations from wardrobe items and `LaundryConstraints`.

This package deliberately does not call an AI model, fetch machine status, or produce the final laundry plan. It gives the planning module a clean memory layer and gives the UI a frequency signal.

### 6.4 Campus Context

`backend/campus/machine_api.py` defines `LaundryMachineClient` and parsing logic for CleverSchool and Haier machine sources. It normalizes towers, machine ids, machine types, machine status, floors, remaining time, provider information, and mock transport behavior.

`backend/campus/context.py` builds `CampusContext` from machine lists, weather, drying context, and pricing rules. It also computes `MachineQueueEstimate` values by machine type. The context layer is responsible for explicit errors when a dormitory name or provider mapping cannot be resolved.

`backend/campus/weather.py` fetches Tsinghua-area current weather from Open-Meteo and validates the shape of the returned weather values.

### 6.5 Laundry Planning

`backend/laundry/planner.py` produces the reference `LaundryPlan`. It owns rule-based decisions such as:

- Selected item validation and urgent item validation.
- Bucket splitting by wash prohibition, dry clean, hand wash, bedding, dark color or color-bleeding risk, light standard loads, and safe mixed loads.
- Machine type selection and nearest-floor preference when the user provides `preferred_machine_floor`.
- Program selection, detergent amount, laundry-bag recommendation, and risk warnings.
- Cost and duration calculations from `CampusContext.pricing_rules`.
- Explicit global warnings for missing wait estimates, waiting-time constraints, and budget constraints.

`backend/laundry/load_estimator.py` supports load-unit estimation and splitting by load target. Dryer assignment is separated through `recommend_drying`; this keeps the wash plan and drying plan explicit.

### 6.6 Report Generation

`backend/reports/generator.py` converts a `LaundryPlan`, optional `DryingPlan`, wardrobe items, and campus context into a `WashReport`. The report explains the plan that already exists. It must not re-split buckets, recalculate machine decisions, fetch new context, or call an AI model.

The report model contains structured `sections`, `action_steps`, `cost_breakdown`, `savings_notes`, and `risk_notes`. This allows the UI to render information directly instead of parsing prose.

# 7 Cross-Stack Data Models

## 7.1 Python and TypeScript Alignment

The project maintains aligned data model layers:

- Python: `backend/shared/models.py`
- TypeScript: `frontend/src/api/types.ts`

The TypeScript data models keep the APK self-contained. They mirror the concepts required by the mobile planning module, campus context, report screen, and wardrobe UI. The Python models remain the reference layer for backend tests and reference behavior.

## 7.2 Core Data Flow

1. **Profile and configuration.** `ProfileScreen.tsx` saves user profile and ModelHub config through `userProfile.ts` and `modelHubConfig.ts`.
2. **Wardrobe state.** `AddClothingScreen.tsx` creates a local wardrobe item through recognition or manual input. `mobileSummary.ts` exposes wardrobe state to screens.
3. **Dirty basket.** The user selects item ids for the current laundry session. Dirty-basket records include item id, added time, and whether the time is known or estimated.
4. **Campus context.** A configured dormitory enables machine and weather context. Queue estimates are grouped by machine type.
5. **Plan.** The planning module receives selected items, constraints, campus context, and pricing rules, then returns buckets, warnings, and estimates.
6. **Execution.** When the user confirms execution, `mobileSummary.ts` writes a completed-laundry record, resets wear counts for completed items, increments wash counts, and clears the dirty basket.
7. **Drying plan and report.** Dryer recommendations and report generation consume the existing plan and context. Report can also show weekly completion history after the basket is empty.

# 8 External Services and Security

## 8.1 External Service Boundaries

| Service | Trigger | Purpose |
|---|---|---|

| | | |
|---|---|---|
| ModelHub / Gemini v1beta | User clicks image recognition, batch recognition, or text recognition after configuring credentials. | Converts image or text evidence into structured clothing fields. |
| CleverSchool | User saves a supported dormitory building. | Reads campus machine towers and machine status for supported devices. |
| Haier | User saves a supported dormitory building. | Reads Haier laundry point and device status for supported devices. |
| Open-Meteo | Mobile summary refresh. | Reads Tsinghua-area weather for drying context and user display. |

### 8.2 Secrets and Local Configuration

The mobile app does not ship a real API key. The user enters ModelHub `baseUrl`, `apikey`, and `model_name` in Profile. The app stores this configuration locally and provides a clear action to remove it.

The Python reference implementation reads local ModelHub configuration from `config/api_config.json`, which is ignored by Git. The committed template is `config/api_config.example.json`. The repository must not commit real API keys, `.env` files, keystores, signing passwords, or production secrets.

### 8.3 No Silent Business Guessing

The project rule is that missing business data should become an explicit error, status, or unknown value. New code should not silently invent material, color, machine, weather, price, duration, risk, or plan data. Explicit unavailable states are acceptable because they tell the UI and the user what is missing.

## 9 Testing, Build, and Release

### 9.1 Local Verification Commands

```
uv sync
uv run python -m unittest discover -v

cd frontend
npm install
npm test
npm run build
npm run cap:sync
```

For debug APK generation:

```
cd frontend
npm run apk:debug
```

The debug APK is generated under:

```
frontend/android/app/build/outputs/apk/debug/app-debug.apk
```

### 9.2 Automated Checks

`.github/workflows/preview.yml` runs preview checks for pushes or pull requests targeting preview or main branches. It syncs Python dependencies, runs Python unit tests, installs frontend dependencies with `npm ci`, runs frontend tests, and builds the frontend.

`.github/workflows/release-apk.yml` runs on main branch release flow. It verifies Python and frontend tests, reads the release version from `frontend/package.json`, updates Android version metadata, sets up JDK 21 and Android SDK 35, runs Capacitor sync, builds a signed release APK, uploads an artifact, and publishes a GitHub Release.

Release signing should use GitHub Secrets: `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, and `ANDROID_KEY_PASSWORD`. If secrets are absent, CI can create a temporary build key for an installable artifact, but that key is not suitable for stable long-term update continuity.

### 9.3 Test Coverage Scope

The repository contains backend tests for clothing extraction, wardrobe behavior, campus machine parsing, campus context, weather, laundry planning, report generation, and full integration. The frontend contains Vitest tests for API services, screen rendering, app integration, refresh behavior, profile configuration, dirty basket, completed-laundry history and undo, recommended machine highlights, report weekly summaries, machine detail, report screens, and related UI flows.

## 10 Developer Handoff Notes

### 10.1 Where to Add Future Work

- Add UI behavior in `frontend/src/screens/` and shared shell components in `frontend/src/components/`.
- Add mobile business behavior in the focused `frontend/src/api/` service module that owns that behavior.
- Add Python reference behavior in the owning backend package rather than in a flat backend module.
- Add shared Python models only in `backend/shared/models.py`; add frontend types in `frontend/src/api/types.ts` when the APK needs that shape.
- Update `docs/structure_design.md` if module boundaries, public interfaces, configuration behavior, or main workflow change.

### 10.2 Things Not to Do

- Do not add compatibility wrapper modules for old flat backend paths.
- Do not put model request text, parsers, planning rules, storage internals, or report logic directly into screens.
- Do not add hidden remote backend dependencies for normal wardrobe, basket, planning, or report workflows.

- Do not commit secrets, keystores, API keys, or real user data.
- Do not mask API failures by returning guessed business results.

## 11 Current Limitations and Extension Points

The current product already works as a complete mobile demo. The following are natural extension points:

- Account-backed profile, wardrobe, basket, and completed-history sync could be added later, but current storage is local-device oriented.
- A hosted backend could be introduced for multi-device sync or shared campus configuration, but the current product does not require it for normal use.
- Additional ModelHub models can be supported only through explicit model allow-list changes and tests.
- New campus providers should be added behind the campus API/context boundary and should expose user-facing dormitory names, not raw provider ids.
- More report formats can be added by consuming structured `WashReport` data instead of parsing existing report prose.

## 12 Conclusion

WashMate Campus is implemented as a complete mobile product. The frontend is not only a visual layer; it also contains the APK service logic for the user workflow. The Python backend remains useful as the reference design, test reference, and modular expression of the same domain rules. This split gives the product a practical demo surface while preserving clear module ownership, typed data models, testability, and safe integration boundaries.